

List labelling

Jason Tandiary

May 2026

The objective of the research was to find an online list labelling algorithm that leverages predictions better to achieve an amortized cost lower than the current theoretical bound of $O(\log^2 u)$ where u is the maximum error. I started by looking through the 2 papers given by Professor Stein: Online List Labelling with Predictions and Nearly Optimal List Labelling. From here I started by thinking about deterministic algorithms built upon the list labelling with Predictions paper. Some approaches I tried was playing around with the insertion of the algorithm. I started by trying to analyze statistically the new potential insertion methods.

First, I considered the ranges of potential rank of each insertion to each have uniform chance to be a true rank. Then, I tried to ‘chop off’ any ranks that are already deemed impossible by the past insertions. Here I encountered my first problem: If I simply chopped off based on my past insertion, then I will have given too much weight into my past insertions. Then I tried doing it in a probabilistic way, I tried doing something along the lines of first having a joint pdf between all past insertions that are in the range of the new insertion. But I quickly found that this way of insertion became way too computationally heavy very quick. As I would be doing convolution, it was on the scale of $O(k * n^2)$ per insertion. This is way too slow as even the regular insertions which was on the scale of $O(N)$ took a bit for my computer.

Because the Online list labelling with Predictions paper had a section on how deterministic algorithms would at most get the same bound as theirs, I went to the nearly optimal list labelling paper for inspiration on the algorithm. After digging into this paper, I found that the bottlenecks for the insertion algorithm, which was that the algorithm divided the insertions into cheap and expensive leaves. This dictates the pdf that they used for the insertion since it had to balance the numbers between cheap and expensive leaves for its tree structure. So, I tried to find a way to leverage predictions so that there is no expensive leaf at any time. But after actually crunching the numbers, I noticed that the rebalancing part of this algorithm itself is already more costly than the bounds given by the Online list labelling with Predictions paper. Therefore I had to abandon the ideas of this paper and go back to try inserting randomization in the Online List Labelling paper. Finally, I settled tried just doing random noise

in the insertion for the exact same algorithm for the online list labelling paper. I tried multiple noise functions until I realized that if the functions have any part where it has greater probability of occurrence, there would be a pile up on insertions in that spot, causing rebalancing to happen more often. This made me think that the optimal function to use is then a uniform choice distribution. I tried implementing it through their publicly available github implementation of the paper. After struggling quite a bit with the code, I finally found the correct places to modify the insertion so that the randomization could be fit in. I tried variants of this insertion before settling in my current algorithm:

1. Let the range of uniform distribution be $\log n$, the potential range of insertion would then be $[r - n, r + n]$ where r is the original insertion position based on the original algorithm
2. In this range, check the previous insertions, if there are numbers bigger than the candidate for insertion, the right side of the window will be adjusted so that it cannot be of higher position than the bigger number. Do the same in reverse for the smaller number.
3. Sample the uniform pdf made from the range that is left, call it noisy slot
4. Take the $\max(\text{left of leaf}, \min(\text{right of leaf}, \text{noisy slot}))$ as the new insertion slot to ensure that the new insertion does not fall off the insertion leaf of the original slot
5. If there is room in the new leaf: insert in noisy slot, else: make room in the leaf and insert in original slot.

The last 2 steps are particularly important as without those 2, I found that the insertion algorithm performs much worse than the original slot. This is caused as there is probability that the new insertion randomly selects slots that are always full and so the algorithm has to make room. As this can happen multiple times in one insertion, I found that capping it at one try then going into the original algorithm still smooths out adversarial inserts but helps against ‘unlucky’ cases. I implemented the new insertion algorithm combined with the learnedLLA structure using the classical packed memory array algorithm and the adaptive packed memory array algorithm. The results in black are for the original learnedLLA insertion algorithm while the ones in green are for the new insertion algorithm.

The final results are as follows:

[1024, 2048, 4096, 8192,] inserts

C = 2 times n size

Dataset 1

LearnedLLA+PMA Amortized Cost = [8.6552734375, 7.9482421875, 5.828857421875, 6.822998046875]

LearnedLLA+APMA Amortized Cost = [7.4150390625, 8.43115234375, 5.622802734375, 6.96435546875]

LearnedLLA+PMA Amortized Cost = [8.6123046875, 7.8193359375, 5.741943359375, 6.7117919921875]

LearnedLLA+APMA Amortized Cost = [8.294921875, 8.8896484375, 5.554931640625, 6.779541015625]

PMA Advantage Avg: 1.3

APMA Advantage Avg: -3.34

Dataset 2

LearnedLLA+PMA Amortized Cost = [9.9931640625, 9.60498046875, 10.013671875, 10.589599609375]

LearnedLLA+APMA Amortized Cost = [7.1435546875, 6.48583984375, 6.651123046875, 8.7674560546875]

DarkGreen LearnedLLA+PMA Amortized Cost = [9.9716796875, 9.49951171875, 9.908935546875, 10.4525146484375]

LearnedLLA+APMA Amortized Cost = [7.19140625, 6.4443359375, 6.22705078125, 8.1314697265625]

PMA Advantage Avg: -0.9

APMA Advantage Avg: -3.4

Dataset 3

LearnedLLA+PMA Amortized Cost = [9.6494140625, 8.3017578125, 13.253173828125, 11.8045654296875]

LearnedLLA+APMA Amortized Cost = [8.4443359375, 7.86279296875, 11.265869140625, 10.6630859375]

DarkGreen LearnedLLA+PMA Amortized Cost = [9.51953125, 8.13134765625, 12.8916015625, 11.483642578125]

LearnedLLA+APMA Amortized Cost = [9.41015625, 7.841796875, 11.849365234375, 10.5723876953125]

PMA Advantage Avg: -2.2

APMA Advantage Avg: 3.8

Dataset 4

LearnedLLA+PMA Amortized Cost = [9.2548828125, 10.44873046875, 9.681884765625, 13.516845703125]

LearnedLLA+APMA Amortized Cost = [7.6884765625, 9.697265625, 8.670654296875, 12.36376953125]

LearnedLLA+PMA Amortized Cost = [9.013671875, 10.2919921875, 8.88427734375, 13.1767578125]

LearnedLLA+APMA Amortized Cost = [8.8759765625, 9.82763671875, 9.05517578125, 10.0267333984375]

PMA Advantage Avg: -3.7

APMA Advantage Avg: 0.58

Dataset 5

LearnedLLA+PMA Amortized Cost = [5.8623046875, 5.62451171875, 6.920166015625, 9.9876708984375]

LearnedLLA+APMA Amortized Cost = [6.173828125, 5.7197265625, 6.929931640625, 9.9412841796875]

LearnedLLA+PMA Amortized Cost = [5.703125, 5.4755859375, 6.6103515625, 9.5714111328125]

LearnedLLA+APMA Amortized Cost = [5.9208984375, 5.595703125, 6.605224609375, 9.541748046875]

PMA Advantage Avg: -3.5

APMA Advantage Avg: -3.7

Averages among all methods and datasets based on insertion numbers 1024 data points:

PMA Advantage Avg: -1.37

APMA Advantage Avg: 7.67

2048 data points:

PMA Advantage Avg: -1.69

APMA Advantage Avg: 1.05

4096 data points:

PMA Advantage Avg: -3.63

APMA Advantage Avg: 0.39

8196 data points: PMA Advantage Avg: -2.51APMA Advantage Avg: -7.49

Overall the new algorithm seem to gain bigger advantage as the number of insertions grow bigger. This might be the smoothing of the randomization helping against data points that pile up near the same area as the old algorithm always insert to the left most of the leaf or the next available slot. So I wanted to highlight this one I recently tried for the biggest data insertion I could reasonably handle: 16384 inserts. Dataset 1:

LearnedLLA+PMA Amortized Cost = [6.58148193359375]

LearnedLLA+APMA Amortized Cost = [7.1015625]

LearnedLLA+PMA Amortized Cost = [6.4339599609375]

LearnedLLA+APMA Amortized Cost = [7.05242919921875]

PMA Advantage: -2.24

APMA Advantage: -0.69

Dataset 2:

LearnedLLA+PMA Amortized Cost = [7.4822998046875]

LearnedLLA+APMA Amortized Cost = [6.58709716796875]

LearnedLLA+PMA Amortized Cost = [7.312744140625]

LearnedLLA+APMA Amortized Cost = [6.3470458984375]

PMA Advantage: -2.27

APMA Advantage: -3.64

Dataset 3:

LearnedLLA+PMA Amortized Cost = [17.34307861328125]

LearnedLLA+APMA Amortized Cost = [14.25335693359375]

LearnedLLA+PMA Amortized Cost = [16.93170166015625]

LearnedLLA+APMA Amortized Cost = [14.012451171875]

PMA Advantage: -2.37

APMA Advantage: -1.69

Dataset 4:

LearnedLLA+PMA Amortized Cost = [14.50799560546875]

LearnedLLA+APMA Amortized Cost = [10.99908447265625]

LearnedLLA+PMA Amortized Cost = [14.19720458984375]

LearnedLLA+APMA Amortized Cost = [12.22869873046875]

PMA Advantage: -2.14

APMA Advantage: 11.18

Dataset 5:

LearnedLLA+PMA Amortized Cost = [11.9105224609375]

LearnedLLA+APMA Amortized Cost = [11.89007568359375]

LearnedLLA+PMA Amortized Cost = [11.5810546875]

LearnedLLA+APMA Amortized Cost = [11.575439453125]

PMA Advantage: -2.77

APMA Advantage: -2.65

Overall, 9/10 tests at the highest insertion shows improvement through using the new algorithm. Since this result is quite consistent, I think that this points towards the smoothing being of help rather than just pure chance. While in the end I did not find a new theoretical bound, empirically it seems that the new algorithm performs better than the original learnedLLA algorithm on big insertions.